

Core 1 - OG Onyx

Architecture cybersecurity and AI integration with Praying Mantis

Technical white paper | Release 0.3.0 | 7 October 2026

Companion edition. Supersedes the Core 1 release candidate 0.2.0 white paper of 7 October 2026.

Chadwick A. Sutton

Aethera Quantum Technologies

Executive summary

Core 1 packages the original OG Onyx report engine with a bounded local execution adapter, an offline visual report, and the Praying Mantis planning companion. Its purpose is to make inspectable computation useful to people and compatible AI-assisted workflows: summarize evidence, compare report snapshots, and explore security-related scenarios without applying changes to a host.

The preserved OG JavaScript engine is 3,089 bytes and exposes three functions: summarize, compare, and suggest. It accepts five Aethera report schemas, produces ten distinct metric names, and selects among six fixed advice outcomes. Praying Mantis is a separate implementation with fixture matching, synthetic firewall classification, VPN configuration planning, and a six-axis heuristic. Its inclusion does not enlarge the original engine's API or supported schemas. [1]

The cybersecurity contribution is a constrained evidence-processing pattern. Selected JSON files cross explicit input boundaries; calculations are performed by known code; output carries an unverified-evidence label; and advice does not automatically authorize or execute an action. This can make a review easier to inspect and reproduce. It does not establish that the submitted evidence is true, that a system is secure, or that a vulnerability has been remediated.

What readers should take away

- Core 1 provides deterministic report arithmetic and fixed decision rules. Website-health staleness also depends on the observation clock.
- An LLM can explain a successful engine result, but its explanation must remain separate from the numbers, provenance, and fixed advice returned by Core 1.
- The adapter reduces specific parsing, resource-use, file-handling, and disclosure risks. It is not a complete security boundary for an untrusted operating system or execution host.
- Praying Mantis reports synthetic or plan-only results. Apply is denied even when an internal capability flag is present; a low heuristic score never enables protection.

Scope of this paper

This paper describes the OG engine and its Node.js adapter and presentation, plus the separately bounded Praying Mantis companion. It is for developers, cybersecurity reviewers, AI tool integrators, and users evaluating the release. The package does not train a model, scan a computer for malware, enforce firewall rules, connect a VPN, or quarantine a file. The browser demo and Python planner are not active protection.

GitHub publication, plugin installation, and activation are separate steps. Publishing source does not establish installed-host compatibility or acceptance in a public plugin directory.

Architecture and data flow

Components with different responsibilities

Component	Responsibility	Important boundary
Original engine	Calculate summaries, differences, and fixed next-test advice	Preserved source; no independent evidence collection
Local runner	Read selected files, enforce bounds, verify core bytes, invoke the engine, return JSON	Read-only report inputs; sanitized errors
Visualizer and renderer	Run the adapter, check its result shape, render derived results and an execution receipt	Explicit new HTML output; no remote chart service
Optional AI host	Select an authorized operation and explain the returned result	Host owns permissions, file access, and any later action
Praying Mantis companion	Classify synthetic scenarios and prepare nonexecuting plans	Separate Python and browser implementations; no host enforcement

OG report data flow

1. An authorized user or host selects an operation and original JSON reports. Report text remains data, never executable instructions.
 2. The runner checks file type, size, and read stability, decodes strict UTF-8, parses JSON, and validates tree and key limits.
 3. The runner verifies the engine's byte length and SHA-256, then executes those exact bytes without a second source-file read.
 4. The engine checks required schema fields and calculates summaries. Comparison and advice use original reports, not caller-supplied summaries.
 5. Successful JSON includes results, the evidence label, engine identity, runtime, and timestamps. Failures return fixed error codes and messages.
 6. The visualizer checks the result and receipt, then creates offline HTML. Advice uses the same summarized snapshot.
- The visualization shows actual engine outputs, before-and-after bars, exact values, advice, and a receipt. Metrics have separate scales; risk scores are not mixed with counts. Rounded labels are marked and exact values retained. The five-schema showcase uses clearly labeled synthetic inputs.

Praying Mantis runs through its own entry points. The local preview links the OG showcase and Mantis planner. Its manual aethera-insight/v1 advisory exchange is a separate format; the OG engine does not consume it. Bundling creates no automatic execution or enforcement channel between them.

What is trusted

The engine, adapter, runtime, and operating system are trusted dependencies. Reports remain untrusted evidence. Hash checks detect changes relative to pinned bytes; they do not authenticate a report author or defend against replacement of both the adapter and expected digest. The VM context hosts trusted code with dynamic code generation disabled. Node explicitly warns that the VM module is not a security mechanism for untrusted code. [2]

Supported schemas and exact semantics

The OG engine accepts five exact schema identifiers. The two learning formats share their metric names, so the total is ten distinct names. These are unchanged by the Praying Mantis addition. Validation is limited to the original engine's requirements; acceptance is not full validation of every possible producer field. [1]

Schema	Metrics returned	Meaning and important qualification
aethera-health-export/v1	pages; activeIssues; stalePages	Counts pages, issue entries with active exactly true, and pages with a finite timestamp older than 120,000 ms at observation
aethera-game-design-handoff/v1	helpfulRatings; funRatings; learningContexts	Sums affirmative rating counts and counts keys in the selected stats object
aethera-game-learning/v1	helpfulRatings; funRatings; learningContexts	Uses the same calculation as the design-handoff format
aethera-iterations/v1	completedSets; blockedSets; maxRisk	Counts result records, counts blocked exactly true, and takes the maximum supplied 0–1 score
aethera-save-export/v1	saveRecords	Counts records whose schema is aethera-save/v1 and whose id is a string

Website health

Each page must expose an issues array. Only an issue whose active field is the boolean true increases activeIssues. A finite page timestamp contributes to stalePages only when the current clock minus that timestamp is strictly greater than 120,000 milliseconds. Missing or nonnumeric timestamps are ignored by the original rule; that omission is not proof of freshness. The adapter rejects nonfinite parsed numeric values. Website health here means page diagnostics, not a medical or endpoint-security assessment.

Learning and design feedback

The engine uses learning when present and truthy, otherwise the top-level report, then reads stats. Each context must provide helpYes, helpNo, funYes, and funNo as nonnegative safe integers no larger than 100,000. Only helpYes and funYes are summed in the returned metrics. No satisfaction rate, confidence interval, sample-bias correction, or learned prediction is calculated. A context count measures records, not independent people.

Iteration and save records

Every iteration result needs a boolean blocked and a finite max between zero and one. completedSets is the number of submitted result records; the engine does not run or verify those sets. maxRisk is the largest submitted score, or zero for an empty results array. It is not a calibrated probability, and zero for an empty report must not be presented as evidence of safety.

Every save record needs the expected record schema and a string identifier. A count does not establish that the saves are intact, unique, restorable, or from the claimed source. Those properties require separate checks.

Comparison and next test advice

A reproducible worked example

The bundled iteration example contains two records before and two after. Before: blocked values false and true, with max values 0.2 and 0.7. After: both blocked values false, with max values 0.1 and 0.3. Running the comparison produces:

Metric	Before	After	After minus before
completedSets	2	2	0
blockedSets	1	0	-1
maxRisk	0.7	0.3	-0.39999999999999997

The last value reflects JavaScript floating-point subtraction. A visual label may round it to approximately -0.4; the exact output remains available. The supported conclusion is that the second report contains fewer blocked records and a lower maximum supplied score. The comparison does not prove that a control improved or that the inputs are accurate.

The engine compares only summaries with identical schema identifiers. The two learning formats cannot be compared with each other merely because their metric names match. The order is always after minus before, so callers must preserve which report is earlier.

Six original advice outcomes

The suggest function is an ordered rule tree. It returns the first applicable fixed message, rather than combining findings or generating a new plan. This table paraphrases the original messages; execution returns their unchanged wording.

Condition in evaluation order	Advice outcome
activeIssues greater than zero	Reproduce an active issue in Firefox, make one isolated fix, and export again; distinguish stale tabs from failures
blockedSets greater than zero	Review blocked risk axes and original inputs, fix the issue, and rerun; do not lower scores merely to clear a gate
funRatings exists and equals zero	Play a mechanic and explicitly rate whether it was fun
funRatings exists and is nonzero	Compare a clue with a direct answer and collect explicit helpfulness and fun ratings
saveRecords exists	Back up a test save and verify the scene and inventory after reload in Firefox
Otherwise	Repeat a browser scenario and collect another report; absence of a recorded issue is not proof of complete functionality

Advice is advisory only. The package does not open Firefox, repair a system, modify a report, change a risk score, or run the proposed experiment. Those actions need a separate decision and appropriate permission.

AI knowledge and responsible integration

Where the knowledge resides

OG Onyx's domain knowledge is explicit in its supported fields, arithmetic, thresholds, and fixed advice. Praying Mantis adds explicit synthetic-policy rules and a fixed heuristic propagation matrix. These can be read and tested directly. There are no trained weights, embeddings, retrieval index, or autonomous learning loop in this release. Stored feedback and manual advisory exchange do not train either component.

For a fixed valid input, fixed code, and the same relevant observation clock, the functions follow fixed rules. The health staleness calculation and execution timestamps make a blanket claim of byte-identical output across time inappropriate. JavaScript numeric precision also remains part of the observable behavior.

How an LLM can use the result

A compatible host can run the command-line interface and give the returned JSON to an LLM for explanation. The model may help a reader understand why blockedSets changed, identify a missing follow-up test, or turn a result into an accessible summary. Such interpretation is additional text and must not be represented as engine output or independent corroboration.

An integration should preserve four distinctions:

1. Input claim: what the submitted report says happened.
2. Computation: what the verified engine calculated from that report.
3. Interpretation: what a person or model infers from the result.
4. Action: what an authorized operator decides to do next.

The evidence label, user-supplied; not independently verified, must remain visible when results are copied into a conversation, report, or dashboard. A confident explanation does not upgrade the evidence status. The execution receipt records the local run; it is not a signed attestation or independent audit.

Content does not grant authority

A report, filename, comment, or generated suggestion cannot authorize a host to install software, disclose files, change settings, publish material, or contact another party. Integrators should use separate process arguments or safe argument quoting, never interpolate report text into a shell command, and provide only the user-selected files needed for the operation.

If an LLM also receives arbitrary free-form source content, indirect prompt injection remains a host-level risk. Core 1's numeric summary path does not turn that content into instructions, but it cannot secure unrelated tools or prompts. OWASP recommends separating instructions from untrusted data, limiting tool privileges, and applying human approval where consequential actions require it. These are integration controls, not a claim of universal prompt-injection prevention. [3]

What the release can teach

The package is a practical example of keeping a small calculation inspectable inside a larger AI workflow. Its value can be evaluated through correctness, reproducibility, usability, and appropriate limits. No comparative benchmark, general intelligence claim, superiority claim, or guarantee of better security outcomes is established by this release.

Praying Mantis planning companion

Praying Mantis 0.2.1-candidate is included at the creator's direction as a Core 1 companion. Its Python lab and browser demo remain distinct from the OG source. Named services, a message bus, journal, capabilities, and risk heuristic are application-level conventions, not an operating system or privileged security boundary.

Function	What the implementation does	What its result cannot establish
Fixture matching	Hashes bounded in-memory bytes and compares the digest with a synthetic fixture signature	Malware detection, a clean-file verdict, scanning coverage, or quarantine
Firewall classification	Applies ordered first-match rules to a supplied TCP or UDP flow; defaults to inbound deny and outbound allow	Effective Windows policy, observed traffic, application policy, or an enforced rule
VPN planning	Checks endpoint, planning mode, and DNS syntax and records an allowed configuration subset	Reachability, identity, a handshake, working routes, DNS-leak protection, or a kill switch
Six-axis heuristic	Propagates six supplied values through a fixed matrix and compares the largest value with 0.65	Calibrated risk, real-world probability, deployment approval, or operational readiness

Python and browser behavior differ

The Python firewall classifier is stateless. The browser demo additionally accepts a user-supplied new or established connection state and records a profile, while using the same demonstration rules across profiles. That supplied state is not measured connection tracking. Results from the two implementations should not be treated as interchangeable host-policy observations.

The fixture is an inert demonstration value, not an official antivirus test file. A match can return a quarantine-plan label, but applied remains false. The CLI's antivirus command operates on its bundled in-memory fixture; it does not select or scan files on the user's computer.

No route from a low score to enforcement

ENFORCE is false, and every apply request raises a permission error, including requests given the internal APPLY capability. The six axes are Launch, Assets, Saves, Boundary, Security, and Portable. With the default six propagation steps and gain 0.55, the fixed matrix produces a review heuristic. A value at least 0.65 marks a blocked axis. Regardless of the score, suite_ready remains false and the result is review-only.

Manual exchange and local records

The aethera-insight/v1 exchange allows fixed source, topic, and result enums; a check count from zero to 10,000; and user-reported and advisory labels. Extra fields are rejected. Import and export are manual. This release removes calls to an unavailable disk-save API; snapshots, backups, and export remain browser-local. Result history and a BroadcastChannel are shared by same-origin pages, so app keys are not an isolation boundary. The Python byte matcher caps input at 10,000,000 bytes and mailboxes at 100 queued messages; the journal is unbounded. Keep secrets out of notes and plans, and retain existing security controls.

Cybersecurity threat model

The relevant assets are evidence confidentiality, result integrity, predictable local resource use, preserved engine identity, and the user's authority over subsequent actions. An input adversary can supply misleading reports or synthetic scenarios. A compromised runtime or attacker with write access to the whole package is outside the protections claimed here. The OG adapter's file and tree limits do not automatically apply to Praying Mantis APIs or browser storage.

Threat or failure	Release control	Residual risk or required practice
Malformed or excessive input	File-byte cap, strict UTF-8, JSON parsing, bounded parsed tree	JSON parsing occurs before tree checks; byte limits are the first resource boundary
Unsafe object keys	Reject <code>__proto__</code> , prototype, and constructor in parsed objects	This is one defensive measure, not universal protection for other code paths
Unexpected file types or unstable reads	Regular-file checks, final-component symlink rejection, descriptor metadata checks	No root-directory confinement; intermediate path components and host filesystem semantics remain relevant
Changed original engine	Exact byte length and pinned SHA-256 checked before execution	Trusted distribution of the adapter and digest is still required
Fabricated or misleading evidence	Persistent unverified-evidence label; no automatic action	Plausible false inputs can still pass validation and yield valid arithmetic
Disclosure in errors or visual output	Fixed sanitized error messages; visualization of derived fields	Derived metrics and timestamps can themselves be sensitive; host logs and retention are separate
HTML or script injection	Escaped presentation values, safe serialized data, restricted output fields	Requires continued review if the renderer or allowed fields change
Accidental overwrite	Explicit .html destination with exclusive file creation	User still chooses an appropriate directory and controls later sharing

Interpret controls narrowly

Input validation should cover both syntax and relevant domain constraints. OWASP's guidance also distinguishes validation from output encoding and other context-specific defenses. Core 1 applies several such controls, while retaining the original engine's limited schema semantics. [4]

The generated HTML is self-contained and does not call a remote chart service. It declares a content security policy intended to block network connections and external resources, while permitting the bundled inline script and styles needed by the report. Browser enforcement was not tested. This is defense in depth for the generated document, not a substitute for a trusted browser or a general browser sandbox.

Praying Mantis adds demonstrations and planning, not active antivirus, endpoint detection and response, network monitoring, or malware remediation. Its internal capabilities and mailbox restrictions are not OS access controls. A lower supplied or heuristic score is not a security finding. Security decisions still require validated evidence, qualified judgment, and appropriate operational controls.

Input limits and known edge cases

The OG adapter makes rejection behavior explicit rather than repairing submitted data silently. The following limits apply to OG JSON-file input. They are not shared package-wide limits or a general guarantee against denial of service. Praying Mantis has its own checks and unbounded structures, including an in-memory journal. [1]

Limit	Release value	Scope
Input bytes	2 MiB per file	Before decoding and parsing
Parsed depth	32	Root is depth zero
Parsed values	50,000	Includes object and array containers
Collection entries	10,000	Per object or array
String or key length	65,536 UTF-16 code units	Each surviving parsed string or key

Malformed UTF-8, malformed JSON, nonfinite parsed numbers, unsafe keys, unsupported report types, invalid required fields, mismatched comparison schemas, and failed core-integrity checks are rejected. The runner sends JSON results to stdout, sanitized errors to stderr, and a nonzero exit code on failure.

Parsing and numeric precision

The adapter uses standard JSON.parse. If an object contains repeated names, the last value is retained. Overwritten values are not independently checked against parsed-tree limits, although the complete file remains subject to the byte cap. Producers should avoid duplicate names, and a stricter duplicate-rejection policy would need an explicit future change. RFC 8259 recommends unique object names and notes the interoperability risks of duplicates and differing numeric precision. [5]

Numbers use JavaScript IEEE-754 arithmetic. Finite values are not automatically exact integers; the original rating fields receive additional safe-integer checks. Floating-point differences should not be silently rewritten as exact decimal values. The example's -0.3999999999999997 illustrates this behavior.

Time and empty reports

Website-health staleness uses the system clock at execution. Reports near the threshold can change classification on a later run even when their bytes do not change. Comparison summarizes each input in turn; a boundary crossing between those summaries is possible. Preserve runtime timestamps and interpret near-threshold results cautiously.

An empty collection can legitimately yield zero metrics under the original rules. Zero means no corresponding items were counted in that input. It does not establish that there were no issues, that a test suite ran, or that an application is safe.

Files and local execution

The runner does not modify input reports. The visualizer writes only the explicitly selected new HTML output and refuses existing destinations. These controls do not restrict all filesystem access by the surrounding host, guarantee atomic snapshots against every hostile filesystem, or replace host permissions. The VM loading timeout applies to source evaluation; it must not be described as a universal timeout for every later engine call.

Validation and reproducibility

Evidence for this version

Fresh local verification used Node.js v24.19.0 and Python 3.12.14. Results for this version are reported separately to preserve what each check measures: [1]

- Node: 212 tests passed, comprising 192 retained OG tests, 17 companion checks, and 3 local-navigation checks.
- Python: all 24 Praying Mantis fixture and regression test methods passed.
- Portable policy checker: 30,048 valid synthetic policy/flow comparisons matched an independently implemented declarative oracle; 86 invalid-input challenges were rejected.

These 7 October 2026 results are source-level evidence, not a security audit, performance benchmark, or proof of host protection. Browser execution was blocked, leaving appearance, mobile layout, and keyboard interaction unverified. Windows, real security-engine, VPN-tunnel, and installed-ChatGPT execution were not tested. Platform and installed-host acceptance remain unverified.

Preserve the original identity

OG Onyx was recovered from Aethera-Quantum-JavaScript-Build.zip at releases/onyx-ai/web/onyx-core.js; inspected duplicate copies matched. Praying Mantis was recovered from releases/praying-mantis/ and carries the previously tested 0.2.1 policy-validation correction. Package version 0.3.0, runner version 0.1.0, and Mantis source version 0.2.1-candidate are distinct. The OG engine has no embedded semantic version.

Engine SHA-256

```
9289fa38751445dce26858585d276ee3c4c61ef809caef1ee85262ac115e24c7
```

Praying Mantis Python engine SHA-256

```
7455eebc45b1d2ef498a9d638973f12622958e4ac16460f0ccaed64485d1b928
```

The Mantis Python and browser planning engines retain their inspected bytes; documentation and the autosave helper have release-specific changes. Digests establish byte identity, not authorship, encryption, report truth, or security. The source manifest records additional component identities.

Reproduce the basic checks

From the repository root, with already-installed runtimes:

```
node --version
node --test
node skills/review-onyx-reports/scripts/run.cjs --version
node skills/review-onyx-reports/scripts/run.cjs compare \
  skills/review-onyx-reports/examples/before.json \
  skills/review-onyx-reports/examples/after.json
node skills/review-onyx-reports/scripts/visualize.cjs \
  showcase --output onyx-showcase.html
python3 -B tests/mantis-policy-checks.py
```

Then run the retained unit suite from the companion directory, where its local imports resolve:

```
cd skills/review-mantis-plans/resources/praying-mantis
python3 -B -m unittest discover -s tests -v
python3 -B cli.py antivirus
```

Use a new HTML filename and return to the root before root-relative commands. Inspect both views through preview/index.html. Preserve the source revision, runtime, input identities, and test log. NIST's SSDF provides broader release-integrity guidance; no conformance claim is made here. [6]

Deployment limits and next steps

Compatibility and release readiness

A host needs authorized file access and the relevant runtime: Node.js for OG Onyx and Python for the Praying Mantis CLI. Other LLM products can use these entry points only when their hosts provide those capabilities. Importing a skill does not demonstrate execution. If a runtime or file is unavailable, report that the selected component did not run; do not substitute model-calculated results.

The verified local runtimes are Node.js v24.19.0 and Python 3.12.14; this does not establish universal version or OS support. The companion source states Python 3.10 or newer. Rerun tests and inspect file handling and views in each target environment. Installed-plugin sessions need separate execution, numerical, integrity, error-handling, and visual acceptance checks. GitHub storage does not install a plugin. No hosted service or service-level agreement is supplied.

Limitations and potential future work

Next evaluations include independent security review, OS coverage, input fuzzing, stricter duplicate-key handling, stable-clock fixtures, accessibility checks, and signed provenance. Live antivirus, quarantine, firewall, or VPN adapters would be new work requiring separate design, authorization, and platform validation. These are possible directions, not implemented features or commitments. Preserve the original engine's identity and version changed semantics explicitly.

The creator reports that the original system was encrypted before testing with the intent of preserving functionality and security. The recovered source does not independently verify that process or its effectiveness. This release contains readable JavaScript and is not an encrypted artifact.

Attribution and license

Core 1 - OG Onyx and Praying Mantis are credited to Chadwick A. Sutton, Aethera Quantum Technologies. The combined first-party release, including the Praying Mantis companion, uses the MIT License, with copyright © 2026 Chadwick A. Sutton (Aethera Quantum Technologies). Consult the distributed LICENSE for the permission notice, conditions, and warranty disclaimer. [1]

References

[1] Core 1 version 0.3.0 with Praying Mantis 0.2.1-candidate. Primary sources: onyx-core.js, run.cjs, visualize.cjs, report-view.cjs, mantis_os.py, security-plans.js, companion helpers, README, license terms, examples, and tests. Repository: [Core 1 repository](#).

[2] Node.js. VM executing JavaScript. <https://nodejs.org/api/vm.html>

[3] OWASP. LLM Prompt Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html

[4] OWASP. Input Validation Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

[5] IETF. RFC 8259, The JavaScript Object Notation Data Interchange Format, sections 4 and 6. <https://www.rfc-editor.org/rfc/rfc8259.html>

[6] NIST. SP 800-218, Secure Software Development Framework Version 1.1. <https://csrc.nist.gov/pubs/sp/800/218/final>

External references accessed 7 October 2026. They explain relevant practices and limitations; they do not endorse or certify Core 1.